

Assignment Template

Course Name & Code: AWS

Instructor: Nitha

Assignment Details

Assignment Title: Task #48792 - Application Load Balancer with AutoScaling for a PHP WebApp

Student Information

Name: Sameer Malik

Email: sameermalik.dev@gmail.com



Instructions and Guidelines

Objective

1. Host a simple web application in an AWS Red Hat Enterprise Linux 9 instance.
 2. Configure an Application Load Balancer (ALB) for the instance containing the web application.
 3. Create an Auto Scaling Group (ASG) with scaling policies (CPU Target: 20%) for the application instance.
 4. Attach the Load Balancer with the Auto Scaling Group.
 5. Verify the working of scaling policies by increasing the load manually (Scale-Out) and cooling down (Scale-In).
-

Step 1: Provisioning the Infrastructure

Create a Launch Template with following user data for and initializing the **Auto Scaling Group (ASG)**. This ensures that any instance launched follows the same configuration.

```
#!/bin/bash
```

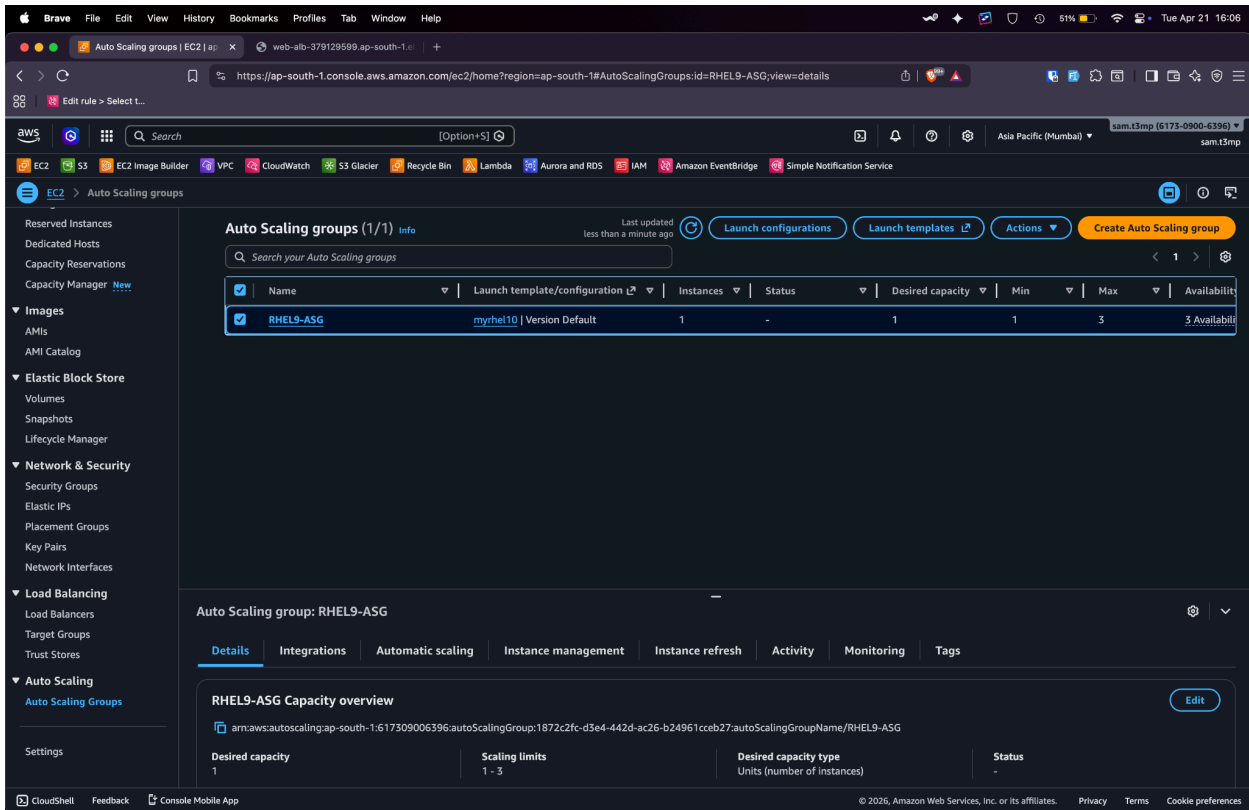
```
sudo dnf install -y httpd
```

```
sudo systemctl enable --now httpd
```

```
echo -e "<p>Host name: <strong>$(hostname)</strong></p>\n<p>IP address:
<strong>$(hostname -I)</strong></p>" | sudo tee /var/www/html/index.html

sudo systemctl restart httpd
```

Auto Scaling Group Configuration

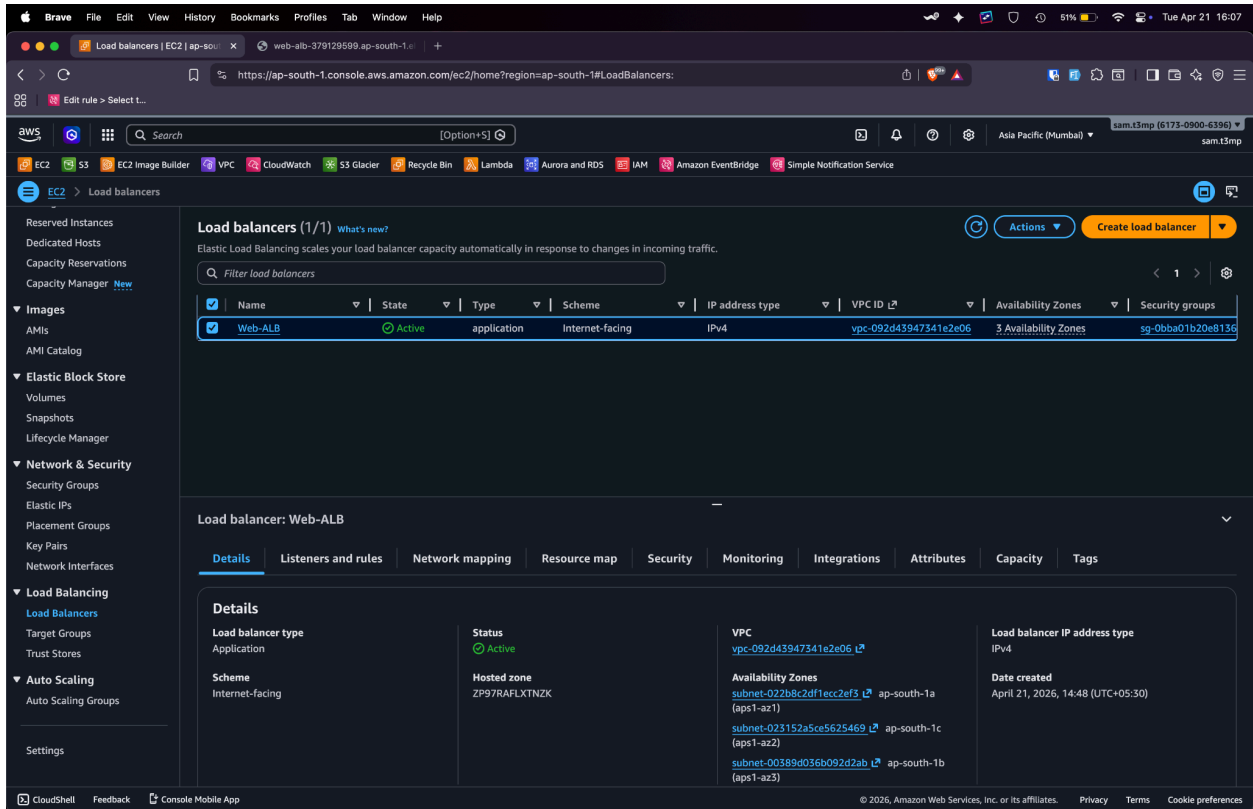


Configuration of RHEL9-ASG showing desired capacity of 1 and max capacity of 3

Step 2: Application Load Balancer (ALB) Setup

A Load Balancer named Web-ALB was configured to distribute incoming traffic across multiple Availability Zones. This provides high availability for the web application.

Load Balancer Overview

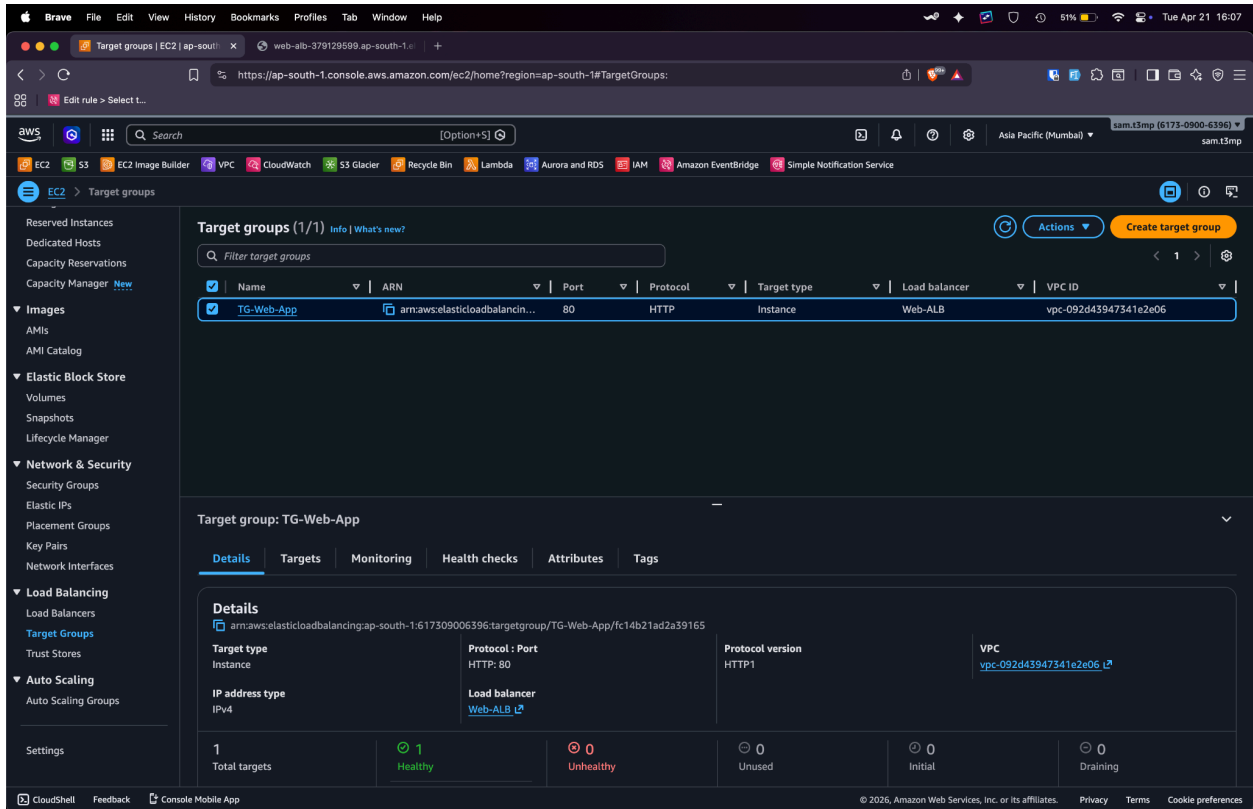


Active Application Load Balancer details showing VPC and Availability Zones

Step 3: Target Group and Health Check Verification

The instances are grouped into a Target Group (TG-Web-App). The Load Balancer performs health checks on the / or /* path to ensure the RHEL10 instances are ready to serve traffic.

Target Group Status

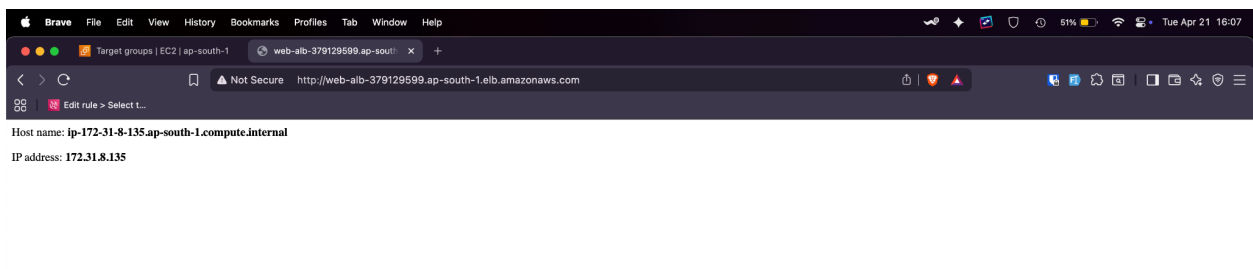


Target Group showing healthy registered instances on Port 80

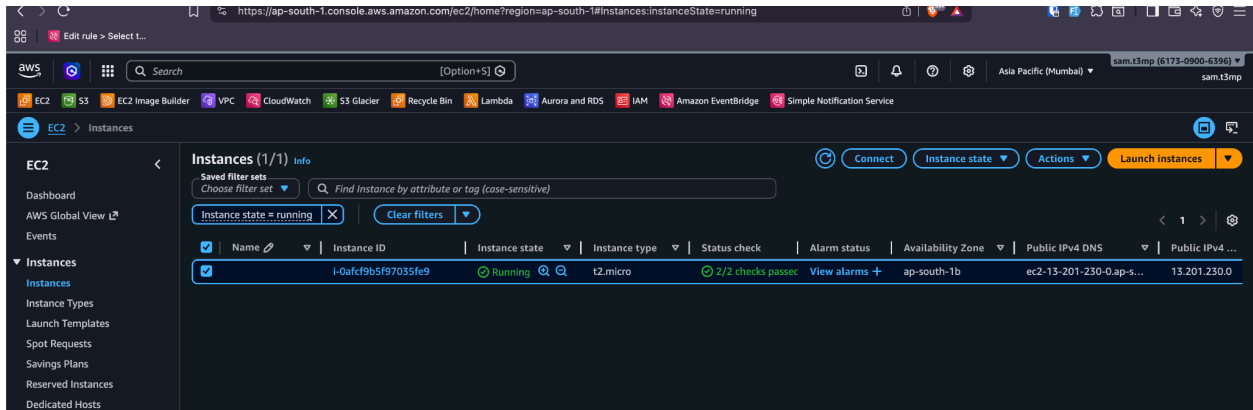
Step 4: Web Application Accessibility

Once the ALB was active, the web application was accessed via the Load Balancer's DNS name. The page displays the private IP and hostname of the specific instance responding to the request.

Initial Web Server Response



Web browser displaying the hostname and private IP



One instance started working

Step 5: Manual Load Testing and Scale-Out

To verify the scaling policy, the stress tool was used on the initial instance to push CPU usage above the **10% threshold**.

Logon to available instance via ssh

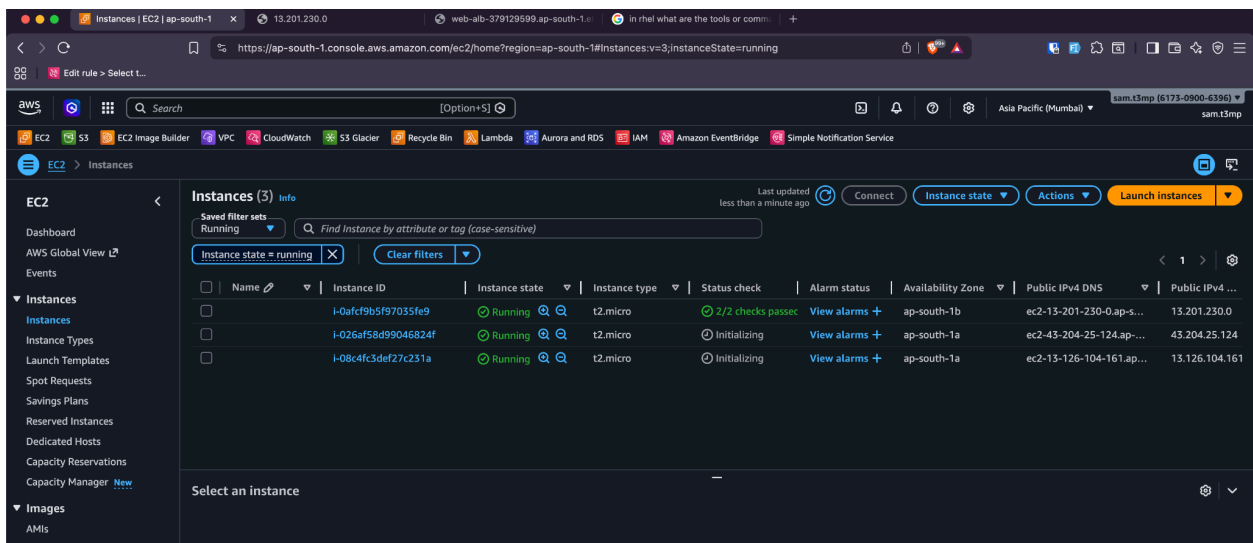
```
sudo dnf install stress-ng
```

```
stress-ng --cpu 0 --metrics-brief &
```

Spam a few of them

Automatic Scale-Out in Progress

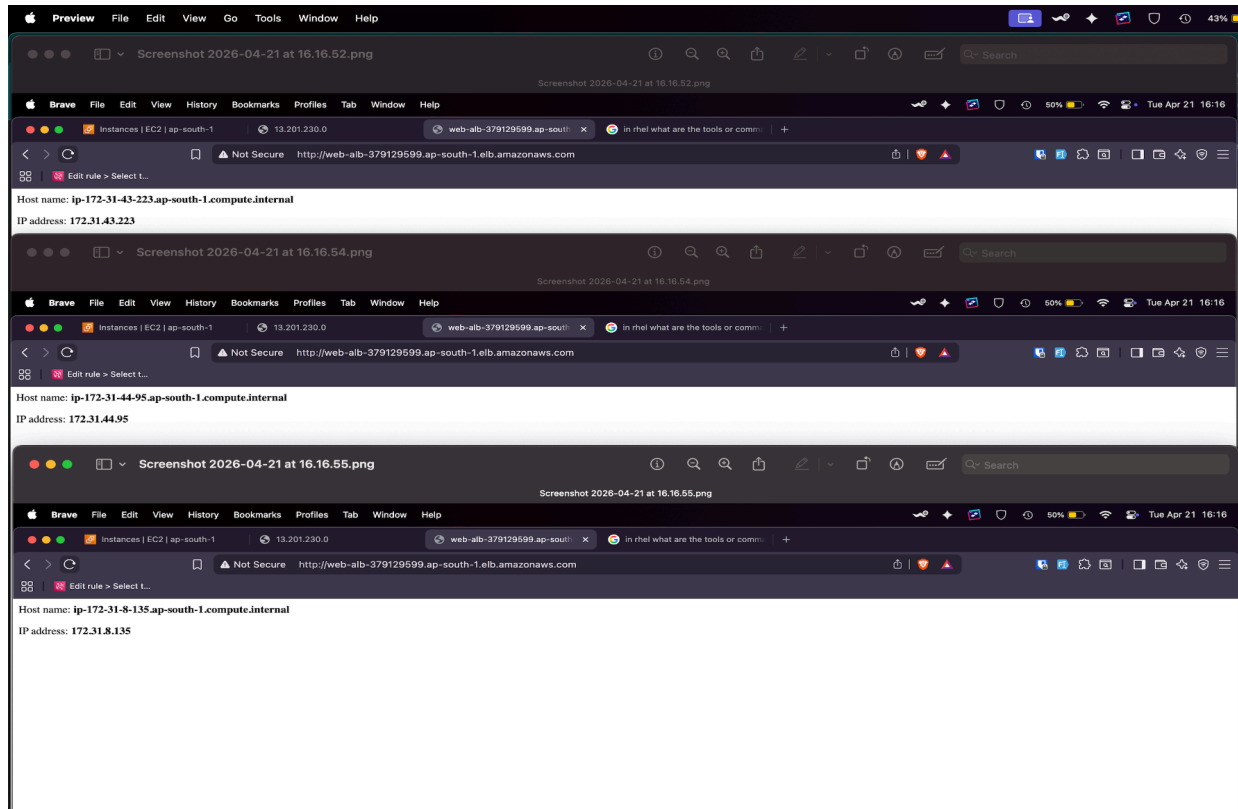
As the CPU load increased, the ASG automatically provisioned two additional instances to handle the demand, bringing the total count to three.



EC2 Dashboard showing three RHEL10 instances running simultaneously during high load

Load Balancer Distribution (URL-Based Verification)

By refreshing the ALB URL, it was confirmed that traffic was being successfully distributed across the three unique hostnames/IP addresses.



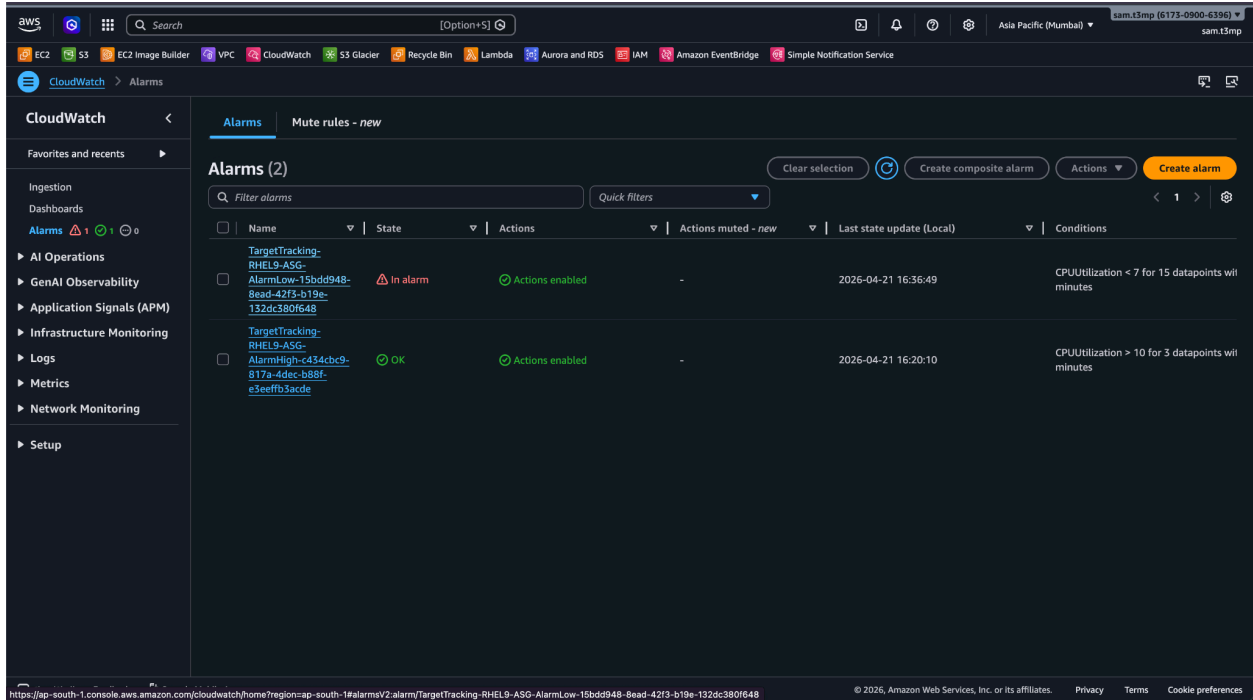
Comparison of three browser tabs showing three different hostnames from the scaled-up instances

Step 6: Verification of Scale-In (Automatic Downscaling)

After the manual stress test was terminated, CPU usage dropped. The CloudWatch alarms detected the low usage and triggered the scale-in policy.

CloudWatch Scaling Alarms

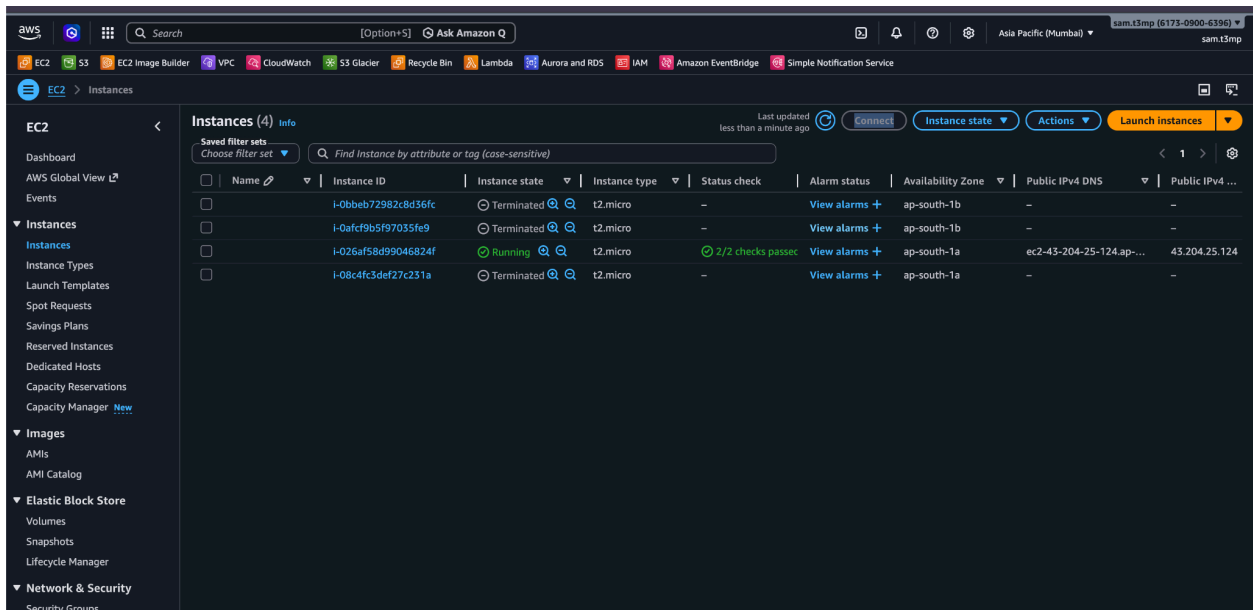
The "AlarmLow" state was triggered when CPU utilization stayed below the threshold for the required duration.



CloudWatch dashboard showing the Low CPU alarm in 'In Alarm' state

Automatic Instance Termination

The ASG automatically terminated the redundant instances once the load subsided, returning the environment to the minimum "Desired Capacity" to save costs.



EC2 Dashboard showing instances in 'Terminated' state after successful scale-in